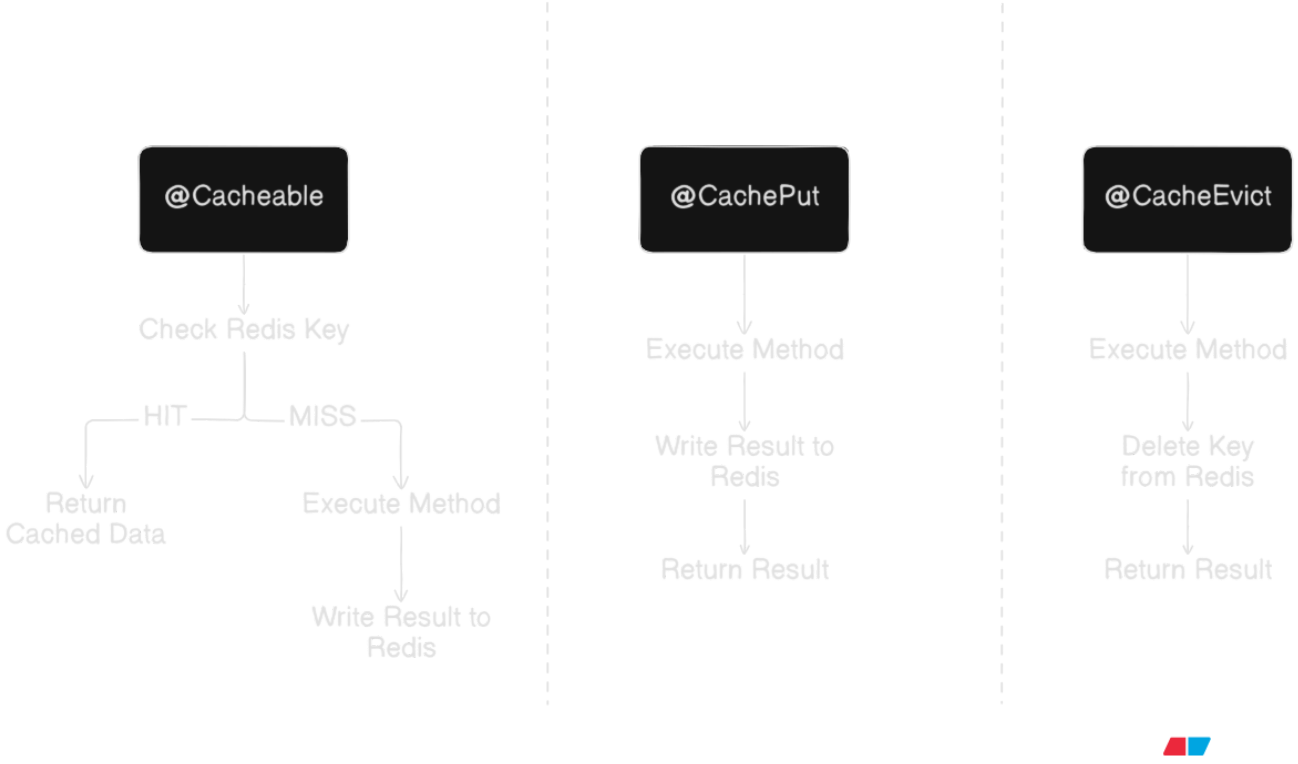
CachePut vs CacheEvict

1. Conceptual Mechanics: `@CachePut` vs `@CacheEvict`

Understanding execution order is critical when designing safe mutation operations.



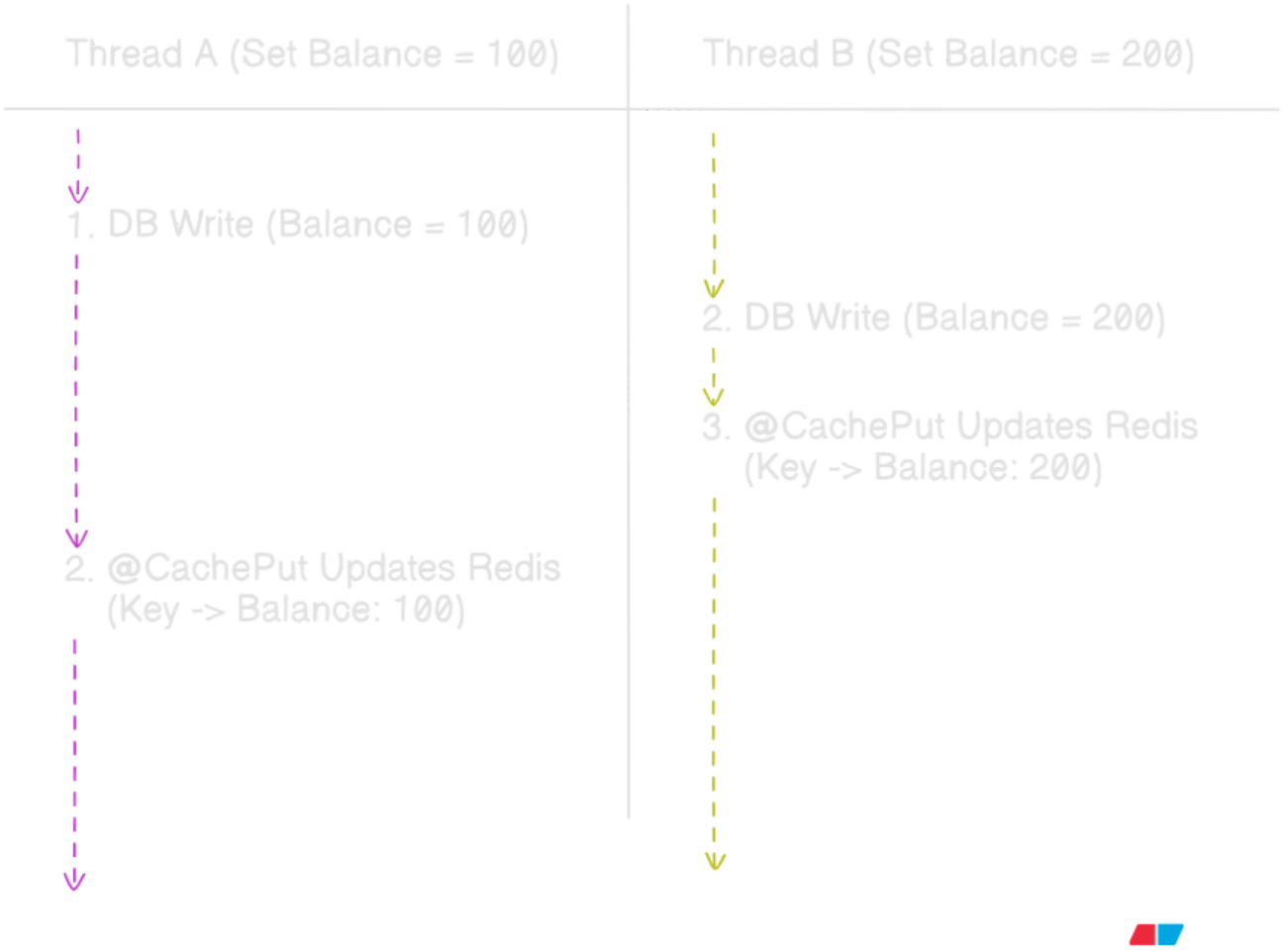
- `@Cacheable` **(Read Pattern):** Skips method execution on cache hit. Writes to Redis on cache miss.
- `@CachePut` **(Update Pattern):** Executes the method **every time**. Writes the method return value directly to Redis under the calculated key.
- `@CacheEvict` **(Delete/Invalidate Pattern):** Executes the method body first, then removes the specified key(s) from Redis (or before execution if `beforeInvocation = true`).

2. High-Concurrency Race Conditions: Why Invalidation Trumps Updates

Updating cached entries via `@CachePut` during DB writes seems intuitive, but in distributed microservices with concurrent threads, it leads to **data inconsistency**.

The `@CachePut` Race Condition Scenario

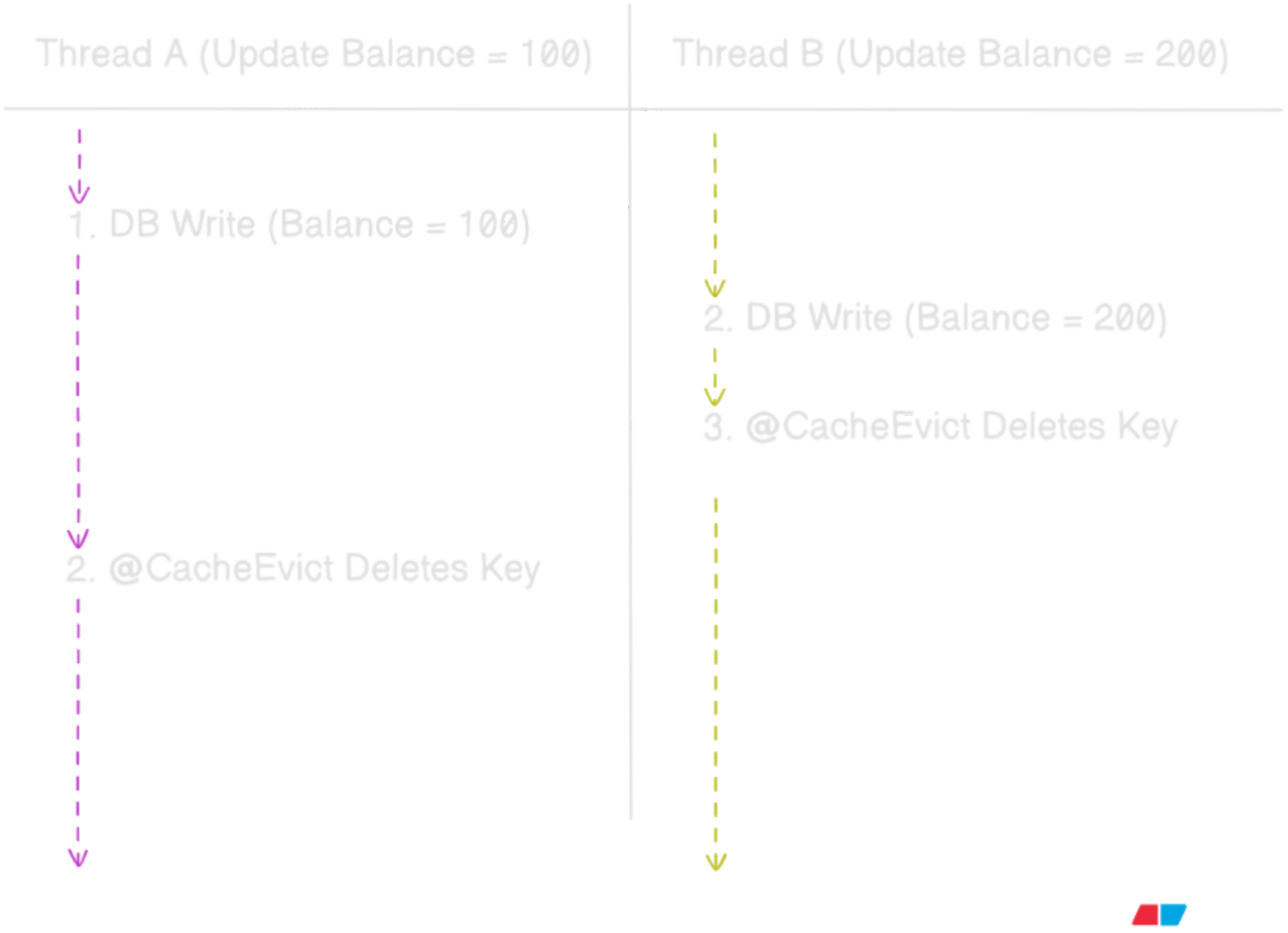
Consider two concurrent HTTP threads (`Thread A` and `Thread B`) updating the same account balance:



- **Database Final State:** `Balance = 200` (Thread B wrote last).
- **Redis Final State:** `Balance = 100` (Thread A's `@CachePut` completed last due to network latency).
- **Result: Silent Data Corruption.** Redis now serves `100` infinitely until TTL expires, completely decoupled from the source of truth.

The Safe Pattern: Eviction over Invalidation

Instead of attempting to keep the cache synchronized via updates, **evict the cache key on mutation** and let the next read repopulate it lazily (Cache-Aside pattern).



- **Redis Final State:** Key is deleted (`Key = null`).
- **Next Read Request:** Triggers a single `@Cacheable` miss, reads the true DB state (`200`), and writes `200` back to Redis safely.

3. Production Eviction Patterns

Pattern A: Single Key Invalidation

Evict a specific entity cache when its state changes.

```
@Service
@RequiredArgsConstructor
public class AccountMutationService {

    private final AccountRepository accountRepository;

    @Transactional
    @CacheEvict(value = "financialSummaries", key = "'user:' + #userId +
':type:' + #accountType")
    public void updateAccountBalance(Long userId, String accountType,
BigDecimal amount) {
        accountRepository.updateBalance(userId, accountType, amount);
```

```
    }
}
```

Pattern B: Bulk Invalidation (`allEntries = true`)

When a operation modifies global state or impacts collections where targeting individual keys via SpEL is impossible, clear the entire cache region.

```
@Service
@RequiredArgsConstructor
public class SystemConfigService {

    @Transactional
    @CacheEvict(value = "exchangeRates", allEntries = true)
    public void refreshExchangeRates(Map<String, BigDecimal> newRates) {
        // Clear all currency pair entries from Redis
        rateRepository.saveAll(newRates);
    }
}
```

Pattern C: Multi-Cache Invalidation (`@Caching`)

When an action alters data spread across multiple distinct cache regions, use `@Caching` to compose multiple evictions into a single annotation call.

```
@Service
@RequiredArgsConstructor
public class UserManagementService {

    @Transactional
    @Caching(evict = {
        @CacheEvict(value = "financialSummaries", key = "'user:' + #userId
+ ':type:ALL'"),
        @CacheEvict(value = "userProfiles", key = "#userId"),
        @CacheEvict(value = "userTransactions", allEntries = true)
    })
    public void deactivateUserAccount(Long userId) {
        userRepository.deactivate(userId);
    }
}
```

Pattern D: Reliable Invalidation (`beforeInvocation`)

By default, `@CacheEvict` executes **after** the method completes successfully. If an exception is thrown inside your service method, eviction is skipped to preserve state.

However, if you need to ensure cache invalidation even if downstream non-critical logic fails, set `beforeInvocation = true`.

```
@CacheEvict(
    value = "sessionTokens",
    key = "#sessionId",
    beforeInvocation = true
)
public void forceLogout(String sessionId) {
    // Session cache is cleared immediately BEFORE executing DB cleanup
steps
    auditLogRepository.logLogout(sessionId);
}
```

4. Implementation Example: Cache Mutation Architecture

Below is a production service managing financial assets using `@Cacheable`, `@CachePut` (for predictable single-threaded overwrites), and `@CacheEvict`.

```
@Service
@Slf4j
@RequiredArgsConstructor
public class FinancialAccountService {

    private final AccountRepository accountRepository;

    // READ: Caches DTO on miss
    @Cacheable(
        value = "financialSummaries",
        key = "'user:' + #userId"
    )
    public FinancialSummaryDTO getSummary(Long userId) {
        log.info("Fetching summary from DB for userId: {}", userId);
        Account account = accountRepository.findByUserId(userId)
                .orElseThrow(() -> new EntityNotFoundException("User not
found"));
        return new FinancialSummaryDTO(userId, account.getBalance(),
account.getStatus());
    }

    // SAFE MUTATION: Evicts key on modification
    @Transactional
    @CacheEvict(
        value = "financialSummaries",
        key = "'user:' + #userId"
    )
    public void depositFunds(Long userId, BigDecimal amount) {
        log.info("Depositing funds in DB for userId: {}", userId);
```

```
        Account account = accountRepository.findByUserId(userId)
                .orElseThrow(() -> new EntityNotFoundException("User not
found"));

        account.setBalance(account.getBalance().add(amount));
        accountRepository.save(account);
        // Cache entry is automatically deleted after this method returns
    }

    // CONTROLLED OVERWRITE: Used only when method returns the updated DTO
directly
    @Transactional
    @CachePut(
        value = "financialSummaries",
        key = "'user:' + #result.userId()"
    )
    public FinancialSummaryDTO createAccount(Long userId, BigDecimal
initialDeposit) {
        log.info("Creating new account in DB for userId: {}", userId);
        Account account = new Account(userId, initialDeposit, "ACTIVE");
        Account savedAccount = accountRepository.save(account);

        // Returns DTO -> @CachePut immediately writes this return value
to Redis
        return new FinancialSummaryDTO(
            savedAccount.getUserId(),
            savedAccount.getBalance(),
            savedAccount.getStatus()
        );
    }
}
```

5. Verification & Redis CLI Inspection

Execution Scenario

```
// 1. Initial Read -> Miss (Fetches DB, stores in Redis)
FinancialSummaryDTO summary1 = accountService.getSummary(2002L);

// 2. Deposit Funds -> Triggers @CacheEvict
accountService.depositFunds(2002L, new BigDecimal("500.00"));

// 3. Subsequent Read -> Miss (DB re-query guarantees fresh data)
FinancialSummaryDTO summary2 = accountService.getSummary(2002L);
```

Inspect Redis state via `redis-cli`

1. **After Initial Read:**

```
127.0.0.1:6379> KEYS "financialSummaries*"
1) "financialSummaries::user:2002"
```

2. **After Invoking `depositFunds()` (`@CacheEvict`):**

```
127.0.0.1:6379> KEYS "financialSummaries*"
(empty array)
```

The key was deleted successfully, eliminating stale reads.

3. **After Second Read:**

```
127.0.0.1:6379> GET "financialSummaries::user:2002"
"
{\"@class\":\"com.example.dto.FinancialSummaryDTO\",\"userId\":2002,\"bala
nce\":1500.00,\"status\":\"ACTIVE\"}"
```

6. Common & Production Pitfalls

1. **SpEL Key Mismatch between `@Cacheable` and `@CacheEvict`:**
   - `@Cacheable(key = "'user:' + #userId")`
   - `@CacheEvict(key = "#userId")` *(WRONG!)*
   - Result: `@CacheEvict` attempts to delete `financialSummaries::2002`, leaving `financialSummaries::user:2002` orphaned in Redis indefinitely.
2. **Transactional Rollback Stale Data:** If `@CacheEvict` (default behavior) runs on method completion, but the wrapping `@Transactional` fails during the DB commit phase, the cache has already been deleted. While safe from serving stale data, it creates an unintended cache miss. Always position eviction after successful transactions.
3. **Evicting Non-Existent Keys:** `@CacheEvict` sends a Redis `DEL` command under the hood. In Redis, `DEL` on a non-existent key returns `0` silently without throwing errors, making typos in key names hard to notice without metric tracking.

Summary

- `@CachePut` **always executes** the underlying method and updates Redis with the new result.
- `@CacheEvict` removes entries from Redis to prevent serving stale data on write/delete operations.
- In high-concurrency systems, using `@CachePut` for database updates introduces **race conditions**; invalidation (`@CacheEvict`) is almost always safer.

- Parameter mismatch across `@Cacheable` and `@CacheEvict` is the #1 cause of silent cache leaks in Spring applications.